Ex 4: Comparative Analysis of Large Language Models

Learning Objective:

Develop a Python program that performs a comparative analysis of Large Language Models by analyzing the text-generation performance of OpenAI, Gemini, and Hugging Face models.

Steps:

1. Import Required Libraries (OpenAI, HuggingFace Transformers, Gemini API)
2. Configure OpenAI key and Google Gemini API key. (This key should be personal. Do not share with anyone).
3. Initialize the Models (OpenAI, Gemini, and Hugging Face).
4. Write function to send prompt and receive response from OpenAI, Gemini, and Hugging Face models
5. Provide a common text prompt. (e.g., “Write a haiku about AI and nature”).
6. Generate Outputs by sending the same prompt to all three models and collect their responses.
7. Display the Model Responses individually. Use Markdown / HTML formatting to display results clearly.
8. Construct an evaluator. (For evaluator you can use Gemini) and produce ranks.
9. Display the ranked model names in best → worst order.

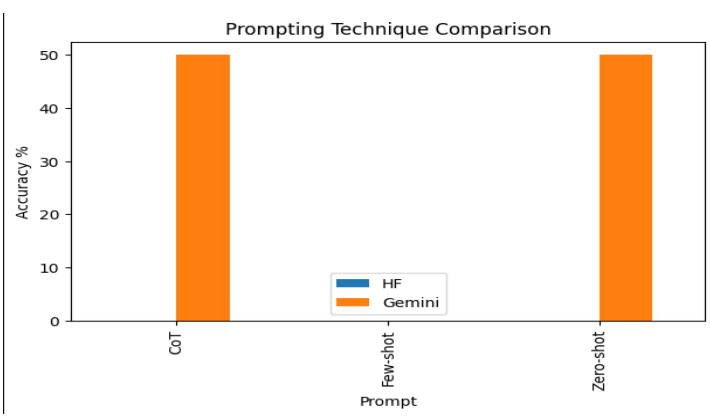
Program:

```
!pip install -q transformers datasets pandas matplotlib google-
generativeai sentencepiece

import re, time
import pandas as pd
import matplotlib.pyplot as plt
from datasets import load_dataset
from transformers import pipeline
import google.generativeai as genai
from getpass import getpass

# Gemini key
genai.configure(api_key=getpass("Enter Gemini API Key: "))

# Small dataset
data = load_dataset("gsm8k", "main")["test"].select(range(2))

# Models
hf = pipeline("text-generation", model="distilgpt2")
gm = genai.GenerativeModel("gemini-2.5-flash")

# Prompts
def zero(q): return f"Q: {q}\nA:"
def few(q): return f"Q: 2+3?\nA: 5\nQ: 10-4?\nA: 6\nQ: {q}\nA:"
def cot(q): return f"Solve step by step.\nQ: {q}\nA:"

# Query functions
def ask_hf(p):
    try:
        return hf(p, max_new_tokens=40, do_sample=False,
pad_token_id=50256)[0]["generated_text"]
    except:
        return "HF Error"

def ask_gm(p):
    try:
        return gm.generate_content(p).text
    except:
        return "Quota exceeded / Try later"

# Number extraction
def get_num(x):
```

```
    n = re.findall(r"-?\d+\.?\d*", str(x).replace(",", ""))
    return n[-1] if n else None

def get_gold(x):
    m = re.search(r"####\s*(-?\d+\.?\d*)", x.replace(",", ""))
    return m.group(1) if m else get_num(x)

# Result check
def check(pred, gold):
    if pred is None:
        return "Hallucination"
    elif pred == gold:
        return "Correct"
    else:
        return "Wrong"

# Run
res = []

for r in data:
    q = r["question"]
    gold = get_gold(r["answer"])

    for name, fn in [("Zero-shot", zero), ("Few-shot", few), ("CoT",
cot)]:
        p = fn(q)

        hf_out = ask_hf(p)
        gm_out = ask_gm(p)
        time.sleep(10)

        hf_pred = get_num(hf_out)
        gm_pred = get_num(gm_out)

        res.append([
            name, q[:50], gold,
            hf_pred, gm_pred,
            check(hf_pred, gold),
            check(gm_pred, gold)
        ])

# Table
df = pd.DataFrame(res, columns=[
    "Prompt", "Question", "Gold", "HF_Pred", "Gemini_Pred", "HF_Result",
"Gemini_Result"
```

```
])

print(df)

# Accuracy
acc = pd.DataFrame({
    "HF": df["HF_Result"].eq("Correct").groupby(df["Prompt"]).mean() *
100,
    "Gemini":
df["Gemini_Result"].eq("Correct").groupby(df["Prompt"]).mean() * 100
})

print("\nAccuracy (%)")
print(acc)

# Plot
acc.plot(kind="bar", figsize=(7,4))
plt.title("Prompting Technique Comparison")
plt.ylabel("Accuracy %")
plt.show()

# Summary
print("\nHF Summary")
print(df["HF_Result"].value_counts())

print("\nGemini Summary")
print(df["Gemini_Result"].value_counts())
```

Output:



Learning Outcome:

Students will be able to integrate and interact with OpenAI, Gemini, and Hugging Face APIs and evaluate model outputs based on relevance.